Task 1. Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

Input Prompt:

Context
You are a senior test lead working in banking domain

Instructions
Create a test strategy document with industry standard
Ask me exactly 5 questions one by one after I answer the previous question for required details

Tone
Professional Tone throughout the document

Output
Create a PDF format file

Chain of thought questions and responses:

Great — I'll gather the required inputs step by step and then generate a complete, industry-standard **Test Strategy Document** in PDF format.

Question 1 of 5

What is the **scope of the application/module** under testing?

(For example: Retail Banking – Payments, Loans, Account Management, Mobile Banking, etc. Also mention key features if possible.)

Retail banking - Payments, key features - UPI payments

Got it — **Scope captured: Retail Banking – Payments (UPI payments)**.

Question 2 of 5

What are the **test environments** involved?

(For example: SIT, UAT, Pre-Prod, Production-like environment, including any details like integrated systems or third-party dependencies such as NPCI, payment gateways, etc.)

SIT, UAT, Pre-PROD, PROD

Noted — Environments: **SIT, UAT, Pre-PROD, PROD**.

Question 3 of 5

What types of **testing scope** should be included?

(For example: Functional, Integration, API, Security, Performance, Usability, Regression, etc. — you can list all that apply.)

Functional, Security, Performance, Regression

Noted — Testing types: **Functional, Security, Performance, Regression**.

Question 4 of 5

What is the **test automation approach**?

(For example: tools/frameworks like Selenium, Playwright, RestAssured, any CI/CD integration, percentage of automation target, etc.)

Manual testing for functional, Selenium automation for regression - 50 % coverage

Captured —

- Functional testing: **Manual**
- Regression: **Selenium Automation (50% coverage target)**

Question 5 of 5

What are the **entry and exit criteria** for testing?

(For example: entry → requirements signed off, build deployed; exit → all critical defects closed, test cases executed, etc.)

industry standard entry and exit criteria

Your **Test Strategy Document (PDF)** has been generated based on the provided inputs and industry standards.

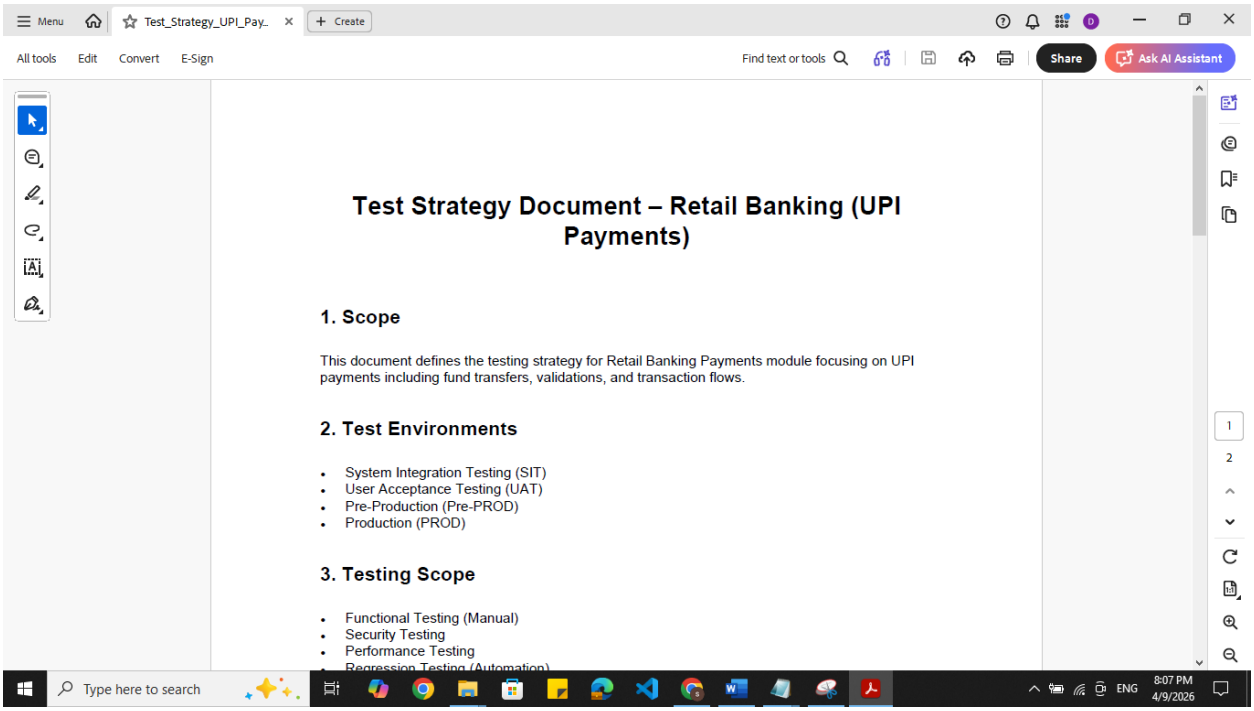
👉 Download the Test Strategy PDF

If you'd like, I can enhance it further by adding:

- Detailed **UPI-specific test scenarios**
- **NPCI integration considerations**
- **Risk-based testing approach**
- **Test data strategy & masking (important for banking)**

Test_Strategy_UPI_Payments.pdf

PDF content:



Test Strategy Document – Retail Banking (UPI Payments)

1. Scope

This document defines the testing strategy for Retail Banking Payments module focusing on UPI payments including fund transfers, validations, and transaction flows.

2. Test Environments

- System Integration Testing (SIT)
- User Acceptance Testing (UAT)
- Pre-Production (Pre-PROD)
- Production (PROD)

3. Testing Scope

- Functional Testing (Manual)
- Security Testing
- Performance Testing
- Regression Testing (Automation)

Menu Test_Strategy_UPI_Pay... Create
All tools Edit Convert E-Sign Find text or tools Share Ask AI Assistant

3. Testing Scope

- Functional Testing (Manual)
- Security Testing
- Performance Testing
- Regression Testing (Automation)

4. Automation Strategy

Regression testing will be partially automated using Selenium with a target coverage of 50%.
Functional testing will be performed manually.

5. Entry Criteria

- Requirements are reviewed and approved
- Test data is available
- Test environment is ready
- Build is deployed successfully
- Test cases are reviewed and approved

6. Exit Criteria

- All planned test cases executed
- All critical and high defects are closed
- Test coverage achieved as per plan
- Test summary report prepared
- Stakeholder sign-off obtained

1
2
Type here to search ENG 8:07 PM 4/9/2026

Menu Test_Strategy_UPI_Pay... Create
All tools Edit Convert E-Sign Find text or tools Share Ask AI Assistant

7. Risks and Mitigation

- Dependency on external systems (UPI/NPCI)
- Environment instability
- Data dependency issues

8. Deliverables

- Test Plan
- Test Cases
- Test Execution Reports
- Defect Reports
- Test Summary Report

2
2
Type here to search ENG 8:08 PM 4/9/2026

Task 2. Design the automation framework structure using a chain-of-thought approach

Output: Analyze and review the generated results

Prompt:

Context

You are a test automation architect working in banking domain

Instructions

Create a keyword driven test automation framework using Selenium with Python, using POM design pattern

Ask me exactly 5 questions one by one after I answer the previous question for required details

Output

Create a PDF format file with generated output

Chain of thought questions and response:

Great — I'll design this step by step to match your exact needs.

Question 1/5:
What type of application are you automating?

- Web application (browser-based)
- Mobile application
- API only
- Hybrid (Web + API, etc.)

Also, if it's a web app, tell me which browsers you need to support (e.g., Chrome, Edge, Firefox).

web app - Chrome only

Got it — Web application with Chrome only 👍

Question 2/5:
What test framework do you want to use with Python?

- `pytest` (most recommended, scalable)

- `unittest` (built-in, simpler)
- `behave` (BDD style – Gherkin)
- Not sure (I can choose best for you)

Let me know your preference.

pytest

Great choice — **pytest** is perfect for scalability and maintainability 🚀

Question 3/5:
How do you want to define your test data and keywords?

- Excel (very common for keyword-driven frameworks)
- JSON
- YAML
- Database
- Not sure (I can recommend)

Let me know your preference.

excel

Perfect — Excel works very well for keyword-driven frameworks 👍

Question 4/5:
What kind of reporting do you want?

- Basic pytest HTML report
- Allure Report (rich, interactive — recommended)
- Custom report (framework-built)
- Not sure (I can suggest best option)

What would you prefer?

basic

Great — we'll go with **basic pytest HTML reporting** 👍

Question 5/5 (final):
Do you want any of the following integrations or features included?

You can choose multiple:

- Logging (log files for execution)
- Screenshot capture on failure

- CI/CD integration (e.g., Jenkins, GitHub Actions)
- Retry mechanism for flaky tests
- Parallel execution

Tell me which ones you want, and I'll generate the complete framework + PDF for you.

logging and screenshots

Your framework PDF is ready.

👉 Download the PDF

If you want, I can next:

- Expand this into full working code (ready-to-run project)
- Add banking-specific validations (transactions, statements, etc.)
- Include CI/CD setup or parallel execution

Just tell me 👍

selenium_keyword_framework.pdf

PDF content:

Keyword Driven Test Automation Framework
(Selenium + Python + Pytest)

Overview

```
Framework Type: Keyword Driven + POM
Language: Python
Automation Tool: Selenium WebDriver
Test Runner: pytest
Browser: Chrome
Data Source: Excel
Reporting: pytest-html
Features: Logging, Screenshot on Failure
```

Folder Structure

```
project/
■
■■■ tests/
■   ■■■ test_runner.py
■
■■■ pages/
■   ■■■ base_page.py
■   ■■■ login_page.py
■
■■■ keywords/
■   ■■■ keyword_engine.py
■
■■■ utils/
```


```
Features: Logging, Screenshot on Failure
```

Folder Structure

```
project/
■
■■■ tests/
■   ■■■ test_runner.py
■
■■■ pages/
■   ■■■ base_page.py
■   ■■■ login_page.py
■
■■■ keywords/
■   ■■■ keyword_engine.py
■
```


```
■■■ utils/
■   ■■■ excel_reader.py
■   ■■■ logger.py
■   ■■■ screenshot.py
■
■■■ config/
■   ■■■ config.py
■
■■■ testdata/
■   ■■■ test_steps.xlsx
■
■■■ conftest.py
```

Sample Keyword Engine

```
def execute_keyword(driver, keyword, locator, value):
    if keyword == "click":
        driver.find_element(*locator).click()
    elif keyword == "input":
        driver.find_element(*locator).send_keys(value)
```

Logging Utility

Menu Test_Strategy_UPI_Payments.pdf selenium_keyword_fra... Create
All tools Edit Convert E-Sign Find text or tools Share Ask AI Assistant

Logging Utility

```
import logging

def get_logger():
    logging.basicConfig(filename='test.log', level=logging.INFO)
    return logging.getLogger()
```

Screenshot Utility

```
def capture_screenshot(driver, name):
    driver.save_screenshot(f"screenshots/{name}.png")
```

Pytest Hook for Screenshot

```
def pytest_runtest_makereport(item, call):
    if call.when == "call" and call.excinfo is not None:
        driver = item.funcargs['driver']
        driver.save_screenshot("failure.png")
```

Type here to search ENG 8:09 PM 4/9/2026

Menu Test_Strategy_UPI_Payments.pdf selenium_keyword_fra... Create
All tools Edit Convert E-Sign Find text or tools Share Ask AI Assistant


```
def capture_screenshot(driver, name):
    driver.save_screenshot(f"screenshots/{name}.png")
```

Pytest Hook for Screenshot

```
def pytest_runtest_makereport(item, call):
    if call.when == "call" and call.excinfo is not None:
        driver = item.funcargs['driver']
        driver.save_screenshot("failure.png")
```


Execution Command

```
pytest --html=report.html
```

Type here to search ENG 8:09 PM 4/9/2026